C语言指针进阶

第八篇Linux、数组、指针、新改与松花粉

1. const 修饰指针

1.1 const修饰变量

变量是可以被改的，如果变量的地址交给一个指针变量，通过指针变量的地址可以间接改这个变量。

上进行实际中基不希望被改的，其实大多数变量，只不过被const修饰后，在语法上加了限制，只要程序员对这种东西一动手动手，就不行你自己玩吧，就报错，程序直接不编译了。

但是如果程序员的心，想改你的话，总有办法一定能实现了，毫无这种情况下就是这么规矩。

```
#include <stdio.h>
int main()
{
    int a = 5;
    a = 20;//这样子是可以改变的
    const int n = 5;
    n = 20;//这样子是不能改变的
    return 0;
}
```

```
#include <stdio.h>
int main()
{
    const int n = 5;
    printf("%d", n);//输出5
    int* p = &n;
    *p = 20;
    printf("%d", n);//输出20
    return 0;
}
```

Microsoft Visual Studio 调试控制台

```
5
20
```

程序运行结果

我们可以看到这里一个有点东西了，但是我们还是看不中一下，为什么这样const修饰呢？就是为了不能被改变，这样子的话，就不能改变原来的东西，这样的话了的话呢？const的原理，这是不清楚的，所以这就让人要新的地址不能改变，那这个东西怎么解决？

1.2 const 修饰指针变量

一般来说const修饰指针变量，可以修饰它的地址，也可以修它的内容，换无数不一样的。

```
int a = 0;//这是一个变量
int const* p = &a;//const 修饰的是指针指向的内容
int * const p1 = &a;//const 修饰的是指针变量
```

我们看下面的，来跟你说一下：

```
#include <stdio.h>
//代码1：修饰指向的内容的情况
void test1()
{
    int a = 10;
    int b = 20;
    int* p = &a;
    *p = 30;//ok
    p = &b;//ok
}
//代码2：修饰const修饰内容的情况
void test2()
{
    int a = 10;
    int b = 20;
    int const* p = &a;
    *p = 20;//ok
    p = &b;//ok
}
//代码3：修饰const修饰内容的情况
void test3()
{
    int a = 10;
    int b = 20;
    int const* const p = &a;
    *p = 20;//ok
    p = &b;//ok
}

int main()
{
    //测试代码1的指向的情况
    test1();
    //测试代码2的指向的内容情况
    test2();
    //测试代码3的指向的内容情况
    test3();
    //测试代码3的指向的内容情况
    test4();
    return 0;
}
```

结论：const修饰指针变量的时候

- 如果放在的左边，那就是修饰的内容，保证指针指向的内容不能通过指针来改变，但是指针变量本身的内容可变。
- const放在的右边，那就是修饰的指针变量本身，保证了指针变量的内容不能改变，但是指针指向的内容，可以通过指针改变。

2. 指针计

新增的指针数组

新指针数组是指针指向的这堆东不同的，**(指针数组，长正数组，变指针数组指针)**

2.1 野指针的成因

- **1. 指针未初始化**

```
#include <stdio.h>
int main()
{
    int *p;//局部变量指针未初始化，默认为随机值
    *p = 20;//野指针的解决
    return 0;
}
```

- **2. 指针越界访问**

```
#include <stdio.h>
int main()
{
    int arr[10] = {0};
    int* p = &arr[0];
    int i = 0;
    for(i = 0; i <= 11; i++)
    {
        //当指向的内容发现超出数组arr的范围时，p就是野指针
        *p++ = i;
    }
    return 0;
}
```

- **3. 指针指向的空间释放**

```
#include <stdio.h>
int* test()
{
    int a = 100;
    return &a;//5.0版本编译时会报警告，运行时结果是无意义的
}

int main()
{
    int* p = test();
    printf("%d\n", *p);
    return 0;
}
```

2.2 如何规避野指针

- **2.2.1 指针初始化**

如果明确知道该指针应该是指向哪个地址，如果不知道该指针应该指向哪里，可以先指针赋值NULL。

NULL是C语言中定义的一个标识符常量，通常，NULL等于0，这个地址是无法使用的，通常是指向空地址。

```
#ifdef __cplusplus
    #define NULL 0
#else
    #define NULL ((void *)0)
#endif
```

初始化如下：

```
#include <stdio.h>
int main()
{
    int num = 10;
    int* p1 = &num;
    int* p2 = NULL;
    return 0;
}
```

- **2.2.2 小心的数组**

一个指针的内存申请了数组空间，通过指针访问数组的时候空间，不能超出范围的话，超出了就是野指针。

- **2.2.3 指针不再使用的时候，使用前检查有效性**

```
int main()
{
    int arr[10] = {1,2,3,4,5,6,7,8,9,10};
    int* ap = &arr[0];
    int i = 0;
    for(i = 0; i < 10; i++)
    {
        *ap++ = i;
    }
    //这时ap已经越界了，可以把ap置为NULL
    ap = NULL;
    //下次使用的时候，判断ap不为NULL的时候再使用
    p = &arr[0];//一定要为ap赋值地址
    if(p != NULL)//一定要
    {
        //...
    }
    return 0;
}
```

- **2.2.4 避免返回局部变量的地址**

如造成野指针的第个指针，不要返回函数内局部指定变量的地址。

3. assert 断言

assert.h

assert.h 头文件定义了宏 assert(int)，用于在运行时确保程序符合指定条件，如果不符合，就报错终止运行。这个宏常常被称为“断言”。

例如，假设可以确定变量 a 一个数的值是为 bbb(a)。

```
assert(a != 0bb(a));
```

上面代码的意思是说：a 的值不能等于 0，或 0bb(a) 。如果确实不等于 0bb(a)，程序继续运行，否则就会终止运行，并且在标准错误输出中打印一条信息。

4. 指针的使用和传址调用

4.1 strlen的模拟实现

库函数 strlen 的功能是求字符串长度，统计字符串中 '\0' 之前的字符的个数，其函数原型如下：

```
size_t strlen ( const char * str );
```

参数strlen是一个字符串的起始地址，然后开始统计字符串中'\0'之前的字符个数，最终返回长度。

如果要模拟实现只要以起始地址不断的往前个字符的偏移，只要不是'\0'字符，计数器每一，这样是即可'\0'就停止。

```
int my_strlen(const char * str)
{
    int count = 0;
    assert(str);
    while(*str)
    {
        count++;
        str++;
    }
    return count;
}

int main()
{
    int len = my_strlen("abcdef");
    printf("%d\n", len);
    return 0;
}
```

4.2 传值调用和传址调用

学习指针的目的是使用指针解决问题，指针小认知，非指针不可呢？

- **4.2.1 传值调用**

形参实参都是可以正常工作的，但是，这样会出现：

strlen函数是形参的拷贝，意思是，这样会出现两个地址，这样两地址的话，它们就可以不同步函数的地址，这样的话，这样的话，这样的话，这样的话，这样的话，这样的话，这样的话，这样的话，这样的话，这样的话，这样的话，这样的话，这样的话，这样的话，这样的话，这

```
void swap(int x, int y)
{
    int tmp = x;
    x = y;
    y = tmp;
}

int main()
{
    int a = 10;
    int b = 20;
    printf("交换前：a=%d b=%d\n", a, b);
    swap(a, b);
    printf("交换后：a=%d b=%d\n", a, b);
    return 0;
}
```

选择 Microsoft Visual Studio 调试控制台

```
交换前：a=10 b=20
交换后：a=10 b=20
```

我们发现其实这个交换函数并没有成功。

swap函数是形参的拷贝，意思它是来本意是要将两个数的了交换，这种调用函数的方式调它之前的函数的地址就加上了，这种的调用方式。

结论：，主参传递的形参的地址，形参会单独创建一份临时空间来保存实参，对形参的修改不影响实参。

所以swap**调失败的了**。

- **4.2.2 传址调用**

我们可以用变量的地址作为函数的参数，在函数内部通过地址（指针）来操作变量，这样的方式称为传址调用。

```
void swap(int* pa, int* pb)
{
    int tmp = 0;
    tmp = *pa;
    *pa = *pb;
    *pb = tmp;
}

int main()
{
    int a = 10;
    int b = 20;
    printf("交换前：a=%d b=%d\n", a, b);
    swap(&a, &b);
    printf("交换后：a=%d b=%d\n", a, b);
    return 0;
}
```

Microsoft Visual Studio 调试控制台

```
交换前：a=10 b=20
交换后：a=20 b=10
```

我们可以看到这里的确实交换成功了，那何时呢？在用，这里的形参swap函数的地址是用变量的地址作为参数了函数，这种函数调用方式称，传址调用。

★ 本节总结

- 指针const修饰不可以修改的可变数指针的地址。
- const修饰指针变量的时候，可以修饰不可变的指针变量，这样指针的内容不可改。
- 野指针的成因：指针未初始化，指针越界访问，指针指向的空间释放，指针不再使用时不置空。
- 断言assert，assert.h文件，assert(int)，如果断言的条件为假，则报错并终止程序运行。
- 传址调用的方式，函数的形参的地址，在函数内部通过地址（指针）来操作变量，这样的方式称为传址调用。